SOFTWARE REQUIREMENTS SPECIFICATION

Courtify

Sports Court Booking & Management Platform

Field	Detail
Document Title	Software Requirements Specification — Courtify
Version	1.0
Status	Final Draft
Date	May 2, 2026
Prepared For	Courtify Development Team
Document Standard	IEEE Std 830-1998
Prepared By	Courtify Engineering

Table of Contents

Revision History **3**
1. Introduction **4**
1.1 Purpose 4
1.2 Document Conventions 4
1.3 Intended Audience and Reading Suggestions 4
1.4 Product Scope 4
1.5 References 5
2. Overall Description **5**
2.1 Product Perspective 5
2.2 Product Functions 5
2.3 User Classes and Characteristics 6
2.4 Operating Environment 6
2.5 Design and Implementation Constraints 6
2.6 User Documentation 7
2.7 Assumptions and Dependencies 7
3. External Interface Requirements **7**
3.1 User Interfaces 7
3.2 Hardware Interfaces 8
3.3 Software Interfaces 8
3.4 Communications Interfaces 8
4. System Features (Functional Requirements) **9**
4.1 User Authentication & Account Management 9
FR-AUTH-01: User Registration 9
FR-AUTH-02: User Login 9
FR-AUTH-03: Logout 10
FR-AUTH-04: Profile Management 10
4.2 Court Management 10
FR-COURT-01: Court Registration (Owner) 10
FR-COURT-02: View Approved Courts (Public) 11
FR-COURT-03: Court CRUD (Owner) 11
FR-COURT-04: Admin Court Status Management 11
4.3 Booking Management 12
FR-BOOK-01: Create Booking 12
FR-BOOK-02: Booking Status Lifecycle 12
FR-BOOK-03: View Bookings 13
FR-BOOK-04: Cancel Booking 13
4.4 Payment Processing 13
FR-PAY-01: Create Stripe Payment Session 13
FR-PAY-02: Stripe Webhook 14
4.5 AI Chatbot (RAG) 14
FR-CHAT-01: Chatbot Query 14
FR-CHAT-02: Chatbot Warmup 14
4.6 Matchmaking 15
FR-MATCH-01: Create Match Request 15
FR-MATCH-02: Browse Match Requests 15
4.7 Reviews & Ratings 15

FR-REVIEW-01: Submit Review 15
FR-REVIEW-02: View Court Reviews 16
4.8 Favourites 16
FR-FAV-01: Manage Favourite Courts 16
4.9 Admin Dashboard 16
FR-ADMIN-01: Platform Statistics 16
FR-ADMIN-02: User Management 17
FR-ADMIN-03: Revenue Report (Owner) 17
5. Other Non-Functional Requirements 17
5.1 Performance Requirements 17
5.2 Safety Requirements 18
5.3 Security Requirements 18
5.4 Software Quality Attributes 18
5.5 Business Rules 19
6. Other Requirements 19
6.1 System Data Model 19
6.2 ENUM Types 20
6.3 System Constraints 20
Appendix A: Glossary 21
Appendix B: API Endpoint Summary 21
Appendix C: To Be Determined (TBD) List 22

Revision History

Name	Date	Reason For Changes	Version
Courtify Engineering	May 2, 2026	Initial SRS Document	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements of Courtify — a web-based sports court booking and management platform. This document is intended for developers, QA engineers, project managers, and stakeholders involved in the design, development, and maintenance of the system.

1.2 Document Conventions

This document follows the conventions defined in IEEE Std 830-1998. Requirements are prefixed with FR- (Functional) or NFR- (Non-Functional). Priority levels are indicated where relevant. Higher-level requirements are assumed to be inherited by detailed sub-requirements unless otherwise noted.

1.3 Intended Audience and Reading Suggestions

This SRS is intended for the following reader types:

- Developers: Read Sections 3 (Functional Requirements), 5 (External Interfaces), and 6 (Data Model) first.
- QA Engineers: Focus on Section 3 (Functional Requirements) and Section 4 (Non-Functional Requirements).
- Project Managers: Begin with Section 2 (Overall Description) and Section 4.
- Stakeholders: Section 2 and Section 1 provide a high-level overview.
- Architects: Sections 2, 5, 6, and 7 cover the technical and constraint landscape.

1.4 Product Scope

Courtify is a full-stack web application that enables players to discover, book, and review sports courts (badminton, squash, futsal, etc.), court owners to register and manage their facilities, and administrators to oversee the entire platform. The system includes:

- A React-based Single Page Application (SPA) frontend
- A Flask RESTful API backend with PostgreSQL database
- Stripe payment gateway integration
- A RAG-based AI chatbot (LangChain + Pinecone + Google Gemini)
- A matchmaking module for finding sports partners

1.5 References

- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications
- Courtify GitHub Repository: zayanahmed04/Courtify_New_Web_MVP
- Flask Documentation — https://flask.palletsprojects.com
- Stripe API Documentation — https://stripe.com/docs
- LangChain Documentation — https://docs.langchain.com

2. Overall Description

2.1 Product Perspective

Courtify is a standalone web platform that acts as a marketplace connecting court owners with players. It is not a component of a larger system. The platform interacts with the following external systems:

External System	Purpose
Stripe	Payment processing for advance booking payments
Pinecone	Vector database for AI chatbot knowledge retrieval
Google Gemini (via LangChain)	LLM for chatbot response generation
HuggingFace Sentence Transformers	Text embeddings for semantic search
PostgreSQL (via SQLAlchemy)	Primary relational database
Cloudinary / External URL	Court image hosting

2.2 Product Functions

The major functions of the system are:

- User registration, login, and profile management (roles: user, court_owner, admin)
- Court registration, management, and admin approval workflow
- Time-slot based court booking with advance payment via Stripe
- Booking lifecycle management: pending → confirmed/rejected → completed/cancelled
- AI-powered chatbot using RAG for platform support queries
- Matchmaking — post/browse requests to find sports partners
- Reviews and ratings for courts
- Favourites — save preferred courts
- Admin dashboard with platform statistics and user/court management
- Revenue reporting for court owners
- Automated booking status scheduler (APScheduler)

2.3 User Classes and Characteristics

User Class	Description	Technical Level	Privileges
Player (User)	Searches and books courts, uses matchmaking and chatbot	General public	Book, review, favourite, cancel
Court Owner	Lists and manages courts, approves/rejects bookings	Business user	CRUD on own courts, manage bookings

User Class	Description	Technical Level	Privileges
Administrator	Platform-wide management and oversight	Internal staff	Full platform access
Unauthenticated Visitor	Browses home page, views court listings	General public	Read-only public data

2.4 Operating Environment

Component	Technology
Frontend	React 18, Vite, TailwindCSS, shadcn/ui
Backend	Python 3.x, Flask 3.1.2, Flask-RESTX 1.3.0
Database	PostgreSQL (via SQLAlchemy 2.0, Flask-Migrate / Alembic)
Authentication	JWT (Flask-JWT-Extended 4.7.1), bcrypt password hashing
Payments	Stripe 14.0.1
AI/RAG	LangChain 0.3, Google Gemini, Pinecone, HuggingFace
Task Scheduling	APScheduler 3.10.4
Deployment	Gunicorn (backend), Vercel (frontend)
Browser Support	Chrome, Firefox, Edge, Safari (latest 2 versions)

2.5 Design and Implementation Constraints

- JWT tokens must be validated on all protected routes before processing requests.
- Booking slots must enforce a database-level UNIQUE constraint on (court_id, booking_date, start_time, end_time).
- Stripe webhooks must be verified using Stripe's HMAC signature validation.
- All passwords must be hashed using bcrypt before storage — plaintext passwords shall never be persisted.
- The RAG chatbot requires a live Pinecone index and active LLM API keys at runtime.
- CORS must be configured to allow only trusted frontend origins.

2.6 User Documentation

The following documentation components will be delivered with the platform:

- User manual covering registration, booking, payment, and chatbot usage.
- Court owner onboarding guide for registration and court management.
- API documentation auto-generated via Flask-RESTX (Swagger UI).
- In-app FAQ and chatbot knowledge base (Markdown files in /rag/knowledge_base/).

2.7 Assumptions and Dependencies

- Users are assumed to have a modern web browser with JavaScript enabled.
- The platform assumes valid Stripe, Pinecone, and Google Gemini API keys are configured via environment variables.

- Court owners are assumed to be verified legitimate businesses.
- The scheduler assumes the server runs continuously to trigger automated status updates.

3. External Interface Requirements

3.1 User Interfaces

The frontend is a React 18 SPA with role-based routing. Key interfaces include:

Page / Component	Description
Home Page (HomePage.jsx)	Landing page with court search, features, FAQ, and chatbot widget
Auth Page (AuthPage.jsx)	Combined login and registration form with role selection
User Dashboard (UserDashboard.jsx)	Player view: search courts, bookings, matchmaking, favourites
Owner Dashboard (OwnerDashboard.jsx)	Owner view: my courts, add court, manage bookings, revenue
Admin Dashboard (AdminDashboard.jsx)	Admin view: stats, all users, all courts, pending approvals
Booking Page (BookingPage.jsx)	Court slot selection and booking confirmation with payment
Profile Page (ProfilePage.jsx)	View and edit user profile details
Success Page (SuccessPage.jsx)	Post-payment confirmation screen
Cancel Page (CancelPage.jsx)	Payment cancellation screen
Chat Widget (ChatWidget.jsx)	Floating AI chatbot accessible from all pages

3.2 Hardware Interfaces

No dedicated hardware interfaces are required. The system operates on standard server infrastructure accessible via web browsers on desktop and mobile devices.

3.3 Software Interfaces

System	Interface Type	Purpose
PostgreSQL	SQLAlchemy ORM over TCP/IP	Primary data persistence
Stripe	REST API (HTTPS)	Payment session creation and webhook events
Pinecone	REST API (HTTPS)	Vector similarity search for chatbot
Google Gemini	LangChain wrapper (HTTPS)	LLM for chatbot answer generation
HuggingFace	sentence-transformers library	Text embedding generation
APScheduler	In-process thread scheduler	Automated booking status updates

3.4 Communications Interfaces

- All client-server communication shall use HTTPS with JSON payloads.
- Authentication tokens shall be transmitted via the HTTP Authorization header (Bearer scheme).
- Stripe webhook events shall be received via HTTP POST with Stripe-Signature header verification.

4. System Features (Functional Requirements)

Requirements are organized by feature module. Each requirement includes its ID, a description, and relevant attributes.

4.1 User Authentication & Account Management

FR-AUTH-01: User Registration

The system shall allow new users to register by providing username, email, password, phone number, gender, and role (user or court_owner). Passwords shall be hashed using bcrypt before storage.

Attribute	Detail
Input	username, email, password, phone_number, gender, role
Output	201 Created + JWT token
Precondition	Email must not already exist in the system
API Endpoint	POST /api/users/register

FR-AUTH-02: User Login

The system shall authenticate users using email and password. On successful login, a JWT token shall be returned for all subsequent authenticated requests.

Attribute	Detail
Input	email, password
Output	200 OK + JWT token + user role
Precondition	User must be registered and active (is_active = true)
API Endpoint	POST /api/users/login

FR-AUTH-03: Logout

The system shall provide a logout endpoint. Token invalidation is handled client-side by clearing the stored JWT.

FR-AUTH-04: Profile Management

Authenticated users shall be able to view and update their profile including username, phone number, gender, and profile image URL.

Attribute	Detail
GET Endpoint	GET /api/users/current
Update Endpoint	PUT /api/users/update
Role-Based Access	Users can only update their own profile

4.2 Court Management

FR-COURT-01: Court Registration (Owner)

Authenticated court owners shall be able to register a new court by providing name, location, type, hourly_rate, opening_time, closing_time, description, and image URL. Newly registered courts shall default to 'pending' status awaiting admin approval.

Attribute	Detail
Required Fields	name, location, hourly_rate, type, opening_time, closing_time
Default Status	pending
API Endpoint	POST /api/courts/register
Authorization	court_owner role required

FR-COURT-02: View Approved Courts (Public)

Any visitor (authenticated or not) shall be able to retrieve a list of all approved courts for browsing.

Attribute	Detail
API Endpoint	GET /api/courts/all/approved
Filter	Status = 'approved' only
Authorization	None required

FR-COURT-03: Court CRUD (Owner)

Court owners shall be able to view, update, and delete their own courts. Owners shall not be able to modify courts belonging to other owners.

Attribute	Detail
GET (owner's courts)	GET /api/courts/byuser
Update	PUT /api/courts/{court_id}
Delete	DELETE /api/courts/{court_id}
Authorization	court_owner role, own courts only

FR-COURT-04: Admin Court Status Management

Administrators shall be able to approve or reject pending courts and view all pending court submissions.

Attribute	Detail
View Pending	GET /api/courts/all/pending
Update Status	PUT /api/admin/update-court-status/{court_id}
Statuses	approved, rejected, pending
Authorization	admin role required

4.3 Booking Management

FR-BOOK-01: Create Booking

Authenticated players shall be able to book a court by selecting a date, start time, and end time. The system shall validate slot availability and prevent double-booking through a unique database constraint. An advance payment amount shall be recorded at booking time.

Attribute	Detail
Input	court_id, booking_date, start_time, end_time, advance_payment
Default Status	pending
Constraint	UNIQUE(court_id, booking_date, start_time, end_time)
API Endpoint	POST /api/bookings/create
Authorization	user role required

FR-BOOK-02: Booking Status Lifecycle

The booking status shall follow this lifecycle: pending → confirmed/rejected (by owner) → completed (auto by scheduler) / cancelled (by user or admin).

Status	Trigger	Actor
pending	Booking created	System
confirmed	Owner approves	Court Owner
rejected	Owner rejects	Court Owner
cancelled	User cancels with reason	Player / Admin
completed	Booking date/time passes	APScheduler (automated)

FR-BOOK-03: View Bookings

Players shall view their own bookings. Court owners shall view all bookings for their courts. Admins shall view all bookings system-wide.

Endpoint	Role	Scope
GET /api/bookings/mybookings	user	Own bookings only
GET /api/bookings/bycourt/{court_id}	court_owner	By specific court
GET /api/bookings/all-with-courts	admin	All bookings

FR-BOOK-04: Cancel Booking

Players shall be able to cancel their own pending or confirmed bookings by providing a cancellation reason.

Attribute	Detail
API Endpoint	PUT /api/bookings/cancel/{booking_id}
Required Field	cancellation_reason
Authorization	user role, own bookings only

4.4 Payment Processing

FR-PAY-01: Create Stripe Payment Session

The system shall integrate with Stripe to process advance booking payments. A checkout session shall be created and the user redirected to Stripe's hosted payment page.

Attribute	Detail
API Endpoint	POST /api/payments/create-session
Payment Methods	stripe, card (ENUM)
Currency	USD (default)
Success Redirect	Frontend /success page
Cancel Redirect	Frontend /cancel page

FR-PAY-02: Stripe Webhook

The system shall expose a webhook endpoint to receive Stripe payment events. On successful payment confirmation, the payment record status shall be updated to 'completed'.

Attribute	Detail
API Endpoint	POST /api/payments/webhook
Security	Stripe HMAC signature header verification
Authorization	Public (Stripe only)

4.5 AI Chatbot (RAG)

FR-CHAT-01: Chatbot Query

The system shall provide an AI-powered chatbot that answers platform-related questions using Retrieval-Augmented Generation. The chatbot shall use Pinecone as the vector store, Google Gemini as the LLM, and HuggingFace sentence transformers for embeddings. Chat history context shall be supported.

Attribute	Detail
API Endpoint	POST /api/chat/ask

Attribute	Detail
Input	user_message, chat_history (optional)
Vector Store	Pinecone
LLM	Google Gemini (via LangChain)
Knowledge Base	Markdown files in /rag/knowledge_base/
Authorization	None (public)

FR-CHAT-02: Chatbot Warmup

The system shall provide a warmup endpoint to pre-load the RAG pipeline on server startup, reducing cold-start latency for the first user query.

Attribute	Detail
API Endpoint	GET /api/chat/warmup
Authorization	None (called on startup)

4.6 Matchmaking

FR-MATCH-01: Create Match Request

Authenticated players shall be able to post a matchmaking request specifying sport type, location, date/time, contact number, and match details.

Attribute	Detail
Input	sport, location, date_time, contact_number, match_details
API Endpoint	POST /api/matchmaking/create
Authorization	user role required

FR-MATCH-02: Browse Match Requests

All authenticated users shall be able to view all available matchmaking requests and their own requests.

Endpoint	Description
GET /api/matchmaking/all	All match requests (all authenticated users)
GET /api/matchmaking/byuser	Current user's match requests only
DELETE /api/matchmaking/{id}	Delete own match request

4.7 Reviews & Ratings

FR-REVIEW-01: Submit Review

Authenticated players shall be able to submit a numeric rating (1–5) for a court they have previously booked.

Attribute	Detail
Input	court_id, rating (integer 1–5)
API Endpoint	POST /api/reviews/rate
Authorization	user role required

FR-REVIEW-02: View Court Reviews

Any user shall be able to retrieve all reviews for a specific court and view their own submitted reviews.

Endpoint	Description
GET /api/reviews/court/{court_id}	All reviews for a court
GET /api/reviews/my-reviews	Current user's reviews

4.8 Favourites

FR-FAV-01: Manage Favourite Courts

Authenticated players shall be able to add courts to their favourites list, view all favourited courts, and remove courts from the list.

Endpoint	Action
GET /api/favourites/all	View all favourited courts
POST /api/favourites/add	Add a court to favourites
DELETE /api/favourites/delete/{court_id}	Remove from favourites

4.9 Admin Dashboard

FR-ADMIN-01: Platform Statistics

Administrators shall be able to view an aggregated dashboard showing total users, total courts, total bookings, and revenue metrics.

Attribute	Detail
API Endpoint	GET /api/admin/dashboard-stats
Metrics	Total users, active courts, bookings count, revenue
Authorization	admin role required

FR-ADMIN-02: User Management

Administrators shall be able to view all registered users and delete any user account, which cascades to delete all related courts, bookings, payments, and reviews.

Attribute	Detail
Delete Endpoint	DELETE /api/admin/delete-user/{user_id}
Cascade Effect	Deletes all related courts, bookings, payments, reviews
Authorization	admin role required

FR-ADMIN-03: Revenue Report (Owner)

Court owners shall be able to generate revenue reports showing income from completed bookings for their courts.

5. Other Non-Functional Requirements

5.1 Performance Requirements

Requirement	Target
API response time (standard endpoints)	< 500ms under normal load
Page initial load time	< 3 seconds on broadband
Concurrent users supported	>= 100 simultaneous users
Chatbot first response (post-warmup)	< 5 seconds
Database query time	< 200ms for standard queries

5.2 Safety Requirements

Not applicable for this platform category. The system handles financial transactions; all payment-related logic must be verified through Stripe's official APIs to prevent fraudulent transactions.

5.3 Security Requirements

- All API endpoints (except public ones) shall require a valid JWT token in the Authorization header.
- Role-based access control (RBAC) shall be enforced: user, court_owner, admin roles with distinct permissions.
- Passwords shall be stored as bcrypt hashes — plaintext passwords shall never be persisted.
- Stripe webhooks shall be validated using Stripe's HMAC signature verification.
- All sensitive configuration (API keys, DB URL, JWT secret) shall be stored in environment variables, never hardcoded.
- CORS shall be restricted to trusted frontend origins only.
- Input validation shall be applied on all API inputs using Pydantic and Flask-RESTX field validators.

5.4 Software Quality Attributes

Reliability & Availability:

- The system shall target 99.5% uptime during business hours.
- Database transactions shall use rollback on failure to prevent partial writes.

- The APScheduler shall be resilient to application restarts.
- The RAG chatbot shall degrade gracefully if Pinecone or LLM API is unavailable.

Scalability:

- The backend shall be deployed via Gunicorn to support multi-worker processing.
- The database shall support connection pooling via SQLAlchemy.
- The frontend shall be deployed on Vercel's CDN for global distribution.

Usability:

- The interface shall be responsive and usable on desktop, tablet, and mobile browsers.
- Navigation shall be accessible within 2 clicks for any primary function.
- Error messages returned by the API shall be human-readable.
- The chatbot UI shall display conversation history within the session.

Maintainability:

- Database schema changes shall be managed through Alembic migrations.
- The codebase shall follow a modular structure separating routes, models, helpers, and services.
- All API endpoints shall be documented via Flask-RESTX (Swagger auto-generation).

5.5 Business Rules

- A court must be admin-approved before it is visible to players.
- Only the court owner may approve or reject bookings for their courts.
- A player may only cancel their own bookings.
- Completed bookings are set automatically by the scheduler and cannot be cancelled retroactively.
- Monetary values shall be stored as NUMERIC(10,2) to prevent floating-point errors.

6. Other Requirements

6.1 System Data Model

Entity	Key Attributes	Relationships
Users	id, username, email, password_hash, role, gender, img_url, is_active	Has many Courts, Bookings, Payments, Reviews, Favourites, Matchmaking
Courts	id, name, location, type, hourly_rate, status, owner_id, opening_time, closing_time	Belongs to Users (owner); Has many Bookings, Favourites, Reviews
Bookings	id, user_id, court_id, booking_date, start_time, end_time, booking_status, advance_payment, total_amount	Belongs to Users and Courts; Has many Payments
Payments	id, booking_id, user_id, amount, payment_method, transaction_id, payment_status, currency	Belongs to Bookings and Users
Matchmaking	id, user_id, sport, location, date_time, contact_number, match_details	Belongs to Users

Entity	Key Attributes	Relationships
Favourites	id, user_id, court_id, created_at	Belongs to Users and Courts
Reviews	id, user_id, court_id, rating, created_at	Belongs to Users and Courts

6.2 ENUM Types

ENUM Name	Values
user_role	user, admin, court_owner
court_status	approved, rejected, pending
booking_status	pending, confirmed, rejected, approved, cancelled, completed
payment_method	stripe, card

6.3 System Constraints

- The system requires PostgreSQL as the database engine — other RDBMS are not supported without migration changes.
- The RAG chatbot is non-functional without valid PINECONE_API_KEY, GOOGLE_API_KEY, and a pre-ingested vector index.
- Stripe payments require STRIPE_SECRET_KEY and STRIPE_WEBHOOK_SECRET environment variables.
- The frontend is built for deployment on Vercel and configured via vercel.json.
- Alembic migrations must be run before first use to create the PostgreSQL schema.

Appendix A: Glossary

Term	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
JWT	JSON Web Token — used for stateless authentication
RAG	Retrieval-Augmented Generation — AI chatbot architecture
SPA	Single Page Application
FR	Functional Requirement
NFR	Non-Functional Requirement
Court Owner	A registered user who lists and manages sports courts
Player / User	A registered user who searches and books courts
Admin	Platform administrator with full management privileges
Booking Slot	A unique court + date + start_time + end_time combination
Pending	Default booking status awaiting owner approval

Term	Definition
RBAC	Role-Based Access Control
ORM	Object-Relational Mapper — SQLAlchemy used here

Appendix B: API Endpoint Summary

Module	Method	Endpoint	Role Required
Auth	POST	/api/users/register	None
Auth	POST	/api/users/login	None
Auth	GET	/api/users/current	Any authenticated
Auth	PUT	/api/users/update	Any authenticated
Courts	POST	/api/courts/register	court_owner
Courts	GET	/api/courts/all/approved	None
Courts	GET	/api/courts/byuser	court_owner
Courts	PUT	/api/courts/{id}	court_owner
Courts	DELETE	/api/courts/{id}	court_owner
Bookings	POST	/api/bookings/create	user
Bookings	GET	/api/bookings/mybookings	user
Bookings	PUT	/api/bookings/cancel/{id}	user
Bookings	PUT	/api/bookings/approve/{id}	court_owner
Bookings	PUT	/api/bookings/reject/{id}	court_owner
Payments	POST	/api/payments/create-session	user
Payments	POST	/api/payments/webhook	None (Stripe)
Chatbot	POST	/api/chat/ask	None
Chatbot	GET	/api/chat/warmup	None
Matchmaking	POST	/api/matchmaking/create	user
Matchmaking	GET	/api/matchmaking/all	Any authenticated
Reviews	POST	/api/reviews/rate	user
Reviews	GET	/api/reviews/court/{id}	None
Favourites	POST	/api/favourites/add	user
Favourites	DELETE	/api/favourites/delete/{id}	user
Admin	GET	/api/admin/dashboard-stats	admin
Admin	DELETE	/api/admin/delete-user/{id}	admin
Admin	PUT	/api/admin/update-court-status/{id}	admin

Appendix C: To Be Determined (TBD) List

The following items remain to be determined and should be resolved before final release:

- TBD-01: Email notification mechanism for booking status changes (owner/player notifications).
- TBD-02: Refund policy and Stripe refund integration for cancelled confirmed bookings.
- TBD-03: Multi-language (i18n) support scope and target locales.
- TBD-04: Mobile native application (iOS/Android) roadmap and timeline.